the_m_word/introspection.rb
```
class Greeting
  def initialize(text)
    @text = text
  end

  def welcome
    @text
  end
end

my_object = Greeting.new("Hello")
```

I defined a Greeting class and created a Greeting object. I can now turn to the language constructs and ask them questions.

```
my_object.class                          # => Greeting
```

I asked my_object about its class, and it replied in no uncertain terms: "I'm a Greeting." Now I can ask the class for a list of its instance methods.

```
my_object.class.instance_methods(false)     # => [:welcome]
```

The class answered with an array containing a single method name: welcome. (The false argument means, "List only instance methods you defined yourself, not those ones you inherited.") Let's peek into the object itself, asking for its instance variables.

```
my_object.instance_variables                # => [:@text]
```

Again, the object's reply was loud and clear. Because objects and classes are first-class citizens in Ruby, you can get a lot of information from running code.

However, this is only half of the picture. Sure, you can read language constructs at runtime, but what about *writing* them? What if you want to add new instance methods to Greeting, alongside welcome, while the program is running? You might be wondering why on earth anyone would want to do that. Allow me to explain by telling a story.

The Story of Bob, Metaprogrammer

Bob, a newcomer to Ruby, has a grand plan: he'll write the biggest Internet social network ever for movie buffs. To do that, he needs a database of movies and movie reviews. Bob makes it a practice to write reusable code, so he decides to build a simple library to persist objects in the database.